

Debugging

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Software hat (immer) Fehler

- **Testen** = Aufdecken/Provozieren von Fehlern
- **Debuggen** = Finden der Stelle im Code

Warum Debuggen?

Typische Fehlerarten:

- **Exceptions:** Programm bricht ab
- **Logikfehler:** Programm läuft durch, Ergebnis ist aber falsch

Warum Debugger statt `IO.println`:

- **Alle Variablenwerte** in einem Zustand sichtbar
- Programm **Schritt für Schritt** ausführbar
- **Call Stack** (Wer hat wen aufgerufen?)
- Komplexe Bedingungen (z.B. **bedingte Breakpoints**)

Die wichtigsten Debugger-Werkzeuge

The screenshot displays the IntelliJ IDEA IDE interface. The main editor shows the `DebugDemo.java` file with the following code:

```
5 public class DebugDemo {
6
7
8
9     public static void main(String[] args) {
10         int[] numbers = {6, 8, 10};
11
12         // Berechnet die Summe aller geraden Zahlen in der Liste.
13         // Berechnet: " + sum);
14
15         var median = median(numbers);
16         IO.println("Erwarteter Median: 6, berechnet: " + median);
17     }
18
19     /** Summiert alle geraden Zahlen in der Liste. */
20     static int sumEvenNumbers(List<Integer> numbers) { 1 usage
21         var sum = 0;
22         for (int i = 1; i < numbers.size(); i++) {
23             int n = numbers.get(i);
24             if (n % 2 == 0) sum += n;
25         }
26         return sum;
27     }
28
29     /** Berechnet den Median der Liste (vereinfachte Variante für ungerade Anzahl). */
30     static int median(List<Integer> numbers) { 1 usage numbers: size = 5
31         Collections.sort(numbers); numbers: size = 5
32
33         var middle = numbers.size() / 2;
34         return numbers.get(middle);
35     }
36 }
```

A context menu is open over the `main` method, showing the following options:

- Run 'DebugDemo.main()'
- Debug 'DebugDemo.main()' (selected)
- Profile 'DebugDemo.main()' with 'IntelliJ Profiler'
- Modify Run Configuration...

The bottom panel shows the 'Debug' console with the following output:

```
main:14, DebugDemo (debugging)
main:30, DebugDemo (debugging)
```

The 'Threads & Variables' tab is active, showing the state of the `main` thread. The variable `numbers` is expanded, showing its value:

```
@ numbers = (ImmutableCollections$ListN@1011) size = 5
> 0 = (Integer@1015) 2
> 1 = (Integer@1016) 4
> 2 = (Integer@1017) 6
> 3 = (Integer@1018) 8
> 4 = (Integer@1019) 10
```

Standard-Vorgehen beim Debuggen

1. Fehler beobachten

- Was genau passiert? Exception? Falsches Ergebnis?

2. Fehler reproduzierbar machen

- Konkrete Eingaben / Szenario festlegen

3. Hypothese bilden

- Was könnte im Code falsch laufen?

4. Breakpoint setzen

- In einer relevanten Methode / vor einer verdächtigen Stelle

5. Im Debug-Modus starten

- Programm läuft bis zum Breakpoint

6. Schrittweise ausführen & Variablen beobachten

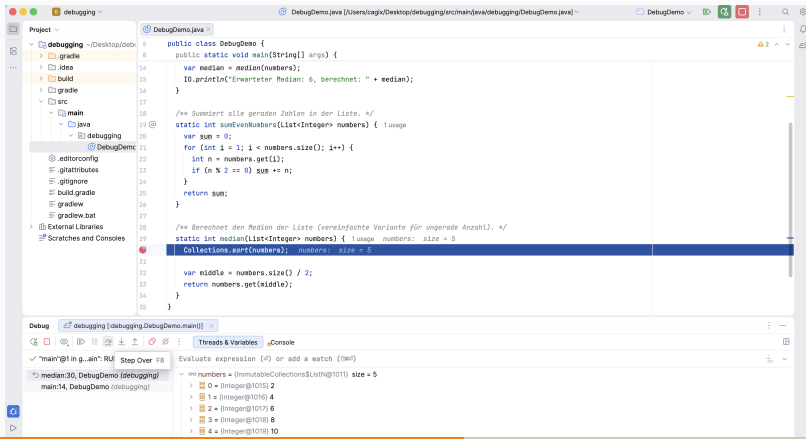
- `Step Over` / `Step Into`, Variablenfenster, ggf. Ausdrucksauswertung

7. Fehler lokalisieren und fixen

- Code korrigieren, dann wieder bei Schritt 1 beginnen

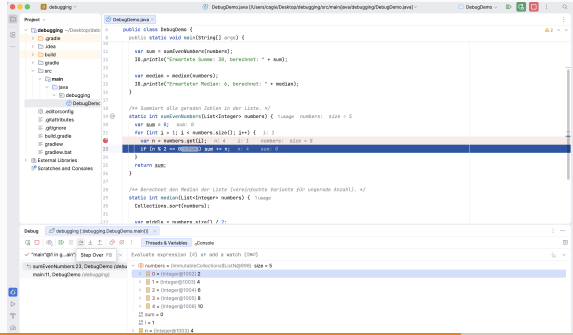
Demo 1: Exception beim Median (Crash)

```
static int median(List<Integer> numbers) {  
    Collections.sort(numbers); // BUG: UnsupportedOperationException  
    var middle = numbers.size() / 2;  
    return numbers.get(middle);  
}
```



Demo 2: Logikfehler bei der Summe (falsches Ergebnis)

```
static int sumEvenNumbers(List<Integer> numbers) {  
    var sum = 0;  
    // BUG: Schleife startet bei 1, Element mit Index 0 (Wert 2) wird nie besucht  
    for (int i = 1; i < numbers.size(); i++) {  
        var n = numbers.get(i); if (n % 2 == 0) sum += n;  
    }  
    return sum;  
}
```



Tipps zum selbstständigen Üben

- Probieren Sie in Ihrer IDE:
 - Einen **Bedingten Breakpoint**: z.B. in der Schleife von `sumEvenNumbers` nur halten, wenn `i == 0` oder `n > 5`
 - Das **Ändern von Variablenwerten** im Debugger
 - Das **Auswerten von Ausdrücken** (Evaluate Expression)
- Ersetzen Sie in `main` die Beispiel-Liste durch andere Daten:
 - z.B. leere Liste, eine Liste mit nur einem Element, ungerade/gerade Anzahl
- Ziel: Sie sollten das Gefühl haben, dass Sie den Debugger **aktiv steuern** können, statt nur “zuzuschauen”.

- Debugger ist zentrales Werkzeug:
 - Breakpoints, Step Over/Into/Out, Variablen, Call Stack
- Vorgehen:
 - Fehler beobachten -> reproduzieren -> Hypothese -> Breakpoint -> schrittweise ausführen -> fixen



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

